Loan Approval Prediction using Random Forest

A binary classification project predicting whether a loan application is **Approved** or **Rejected** from applicant financial and demographic data.

1. Results at a Glance

Metric	Baseline Random Forest	Tuned (GridSearchCV)
Training accuracy	1.0000	0.9968
Test accuracy	**0.9801**	0.9754
5-fold CV accuracy	-	0.9807

Baseline model on the test set (854 samples):

Class	Precision	Recall	F1-score	Support
Rejected (0)	0.98	0.96	0.97	318
Approved (1)	0.98	0.99	0.98	536
Accuracy			**0.98**	854

2. Dataset

File: loan_approval_dataset.csv | **Size:** 4,269 rows x 13 columns | **Missing values:** none | **Duplicates:** none
Target: loan_status (Approved = 2,656, Rejected = 1,613; about 62% / 38%)

Feature	Type	Description
loan_id	int	Unique ID (dropped before training)
no_of_dependents	int	Number of dependents (0-5)
education	categorical	Graduate / Not Graduate
self_employed	categorical	Yes / No
income_annum	int	Annual income
loan_amount	int	Requested loan amount
loan_term	int	Loan term (2-20)
cibil_score	int	Credit score (300-900)
residential_assets_value	int	Value of residential assets
commercial_assets_value	float	Value of commercial assets
luxury_assets_value	float	Value of luxury assets
bank_asset_value	int	Value of bank assets
loan_status	categorical	**Target:** Approved / Rejected

3. Methodology

3.1 Data inspection

head, tail, info, describe, null check and duplicate check. Raw column names contained leading spaces, so they were stripped with `df.columns.str.strip()`, and string values were stripped as well.

3.2 Outlier treatment

Box plots showed outliers in the asset columns. **IQR capping (winsorization)** was applied to luxury_assets_value and commercial_assets_value: values below Q1 - 1.5*IQR or above Q3 + 1.5*IQR were clipped to those bounds.

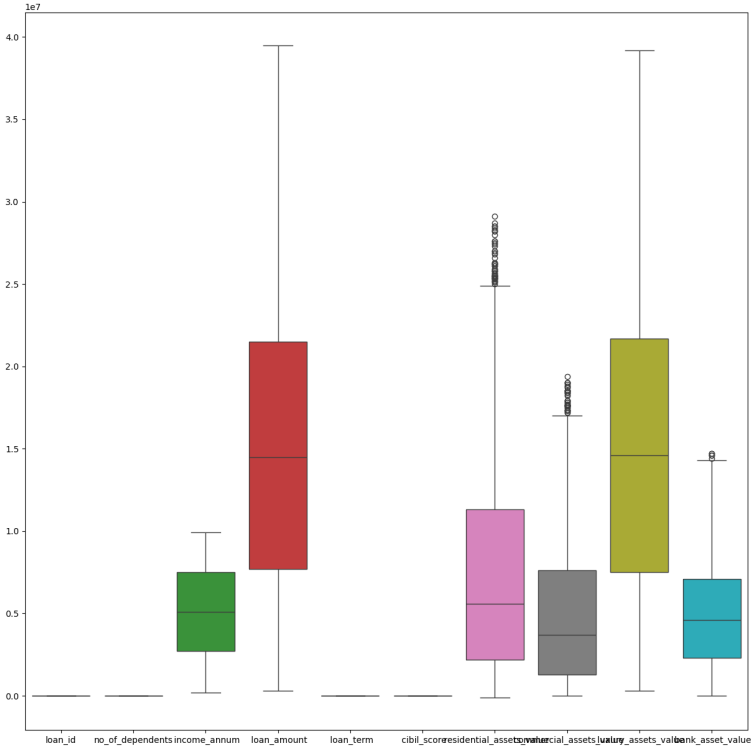


Figure 1: Box plots of numeric features

3.3 Exploratory data analysis

The target classes are moderately imbalanced toward Approved.

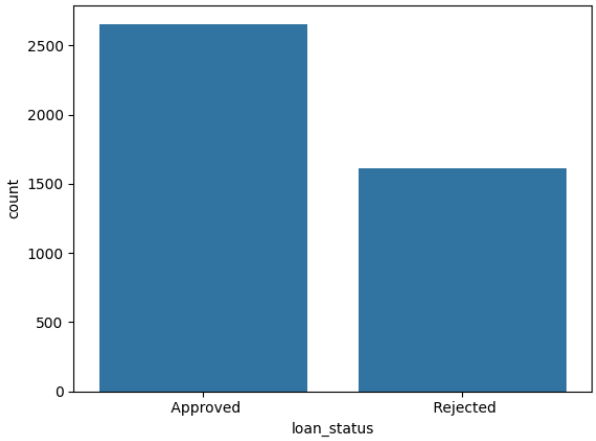


Figure 2: Loan status distribution

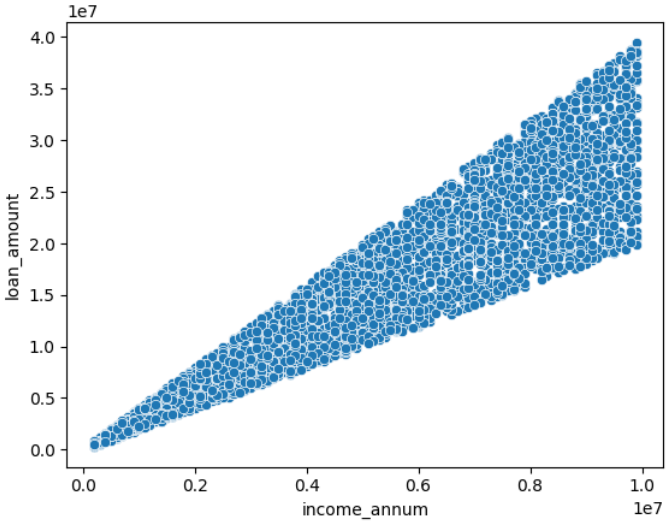


Figure 3: Income vs loan amount

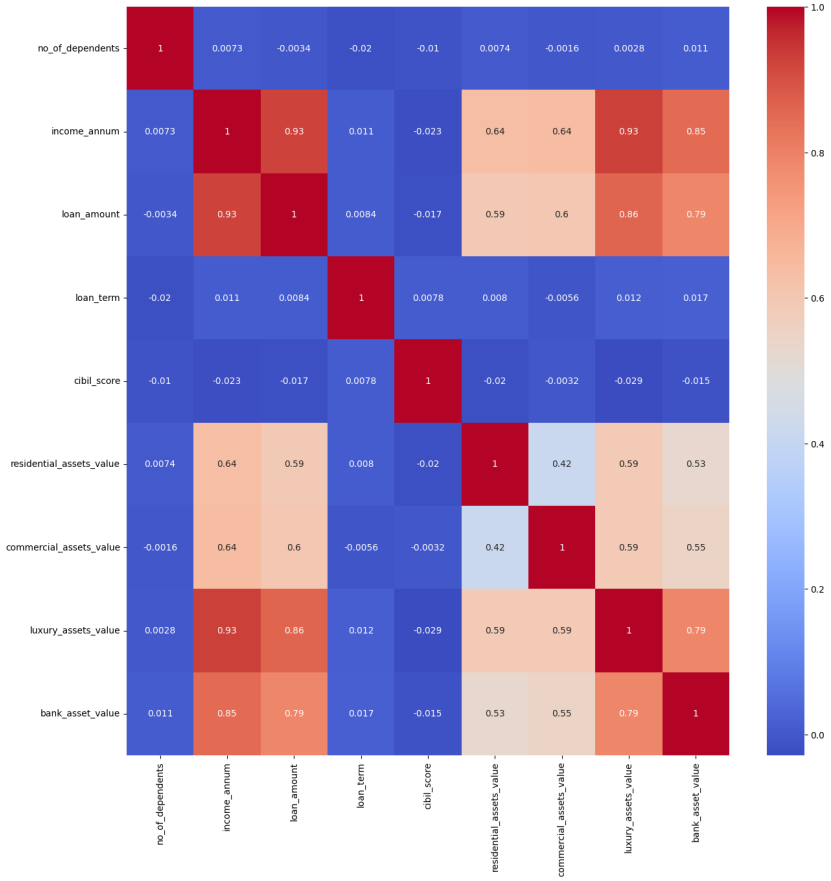


Figure 4: Correlation heatmap

3.4 Encoding

Column	Mapping
education	Graduate = 1, Not Graduate = 0
self_employed	Yes = 1, No = 0
loan_status	Approved = 1, Rejected = 0

3.5 Train/test split

X = all columns except loan_id and loan_status; y = loan_status. 80/20 split with random_state=42 (3,415 train / 854 test).

3.6 Model

RandomForestClassifier() with default parameters.

3.7 Hyperparameter tuning

GridSearchCV with 5-fold cross-validation over 144 combinations (720 fits).

Parameter	Values searched
n_estimators	50, 100, 150, 200
max_depth	None, 10, 20, 30
min_samples_split	2, 5, 10
min_samples_leaf	1, 2, 4

Best parameters: max_depth=20, min_samples_leaf=2, min_samples_split=2, n_estimators=100

4. Evaluation

Confusion matrix (baseline model):

	Predicted Rejected	Predicted Approved
Actual Rejected	306	12
Actual Approved	5	531

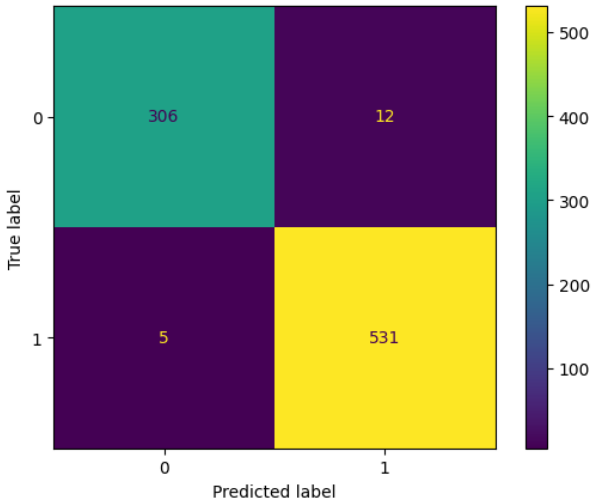


Figure 5: Confusion matrix

Only 17 of 854 test applications were misclassified (12 false approvals, 5 false rejections).

5. Key Findings

- The Random Forest reaches about **98% test accuracy** with balanced precision and recall across both classes.
- **Tuning did not improve test accuracy** (98.01% before vs 97.54% after). The difference is about 4 samples, so the default model is already near the ceiling for this data.
- The baseline has 100% training accuracy against 98% test accuracy, indicating mild overfitting. The tuned model constrains tree depth and leaf size, but the train-test gap stayed about the same (about 2 points).

- The 12 **false approvals** are the costlier error type in real lending, since they are loans granted to applicants who would have been rejected.

6. Limitations and Future Work

- Only two columns were outlier-treated. residential_assets_value has a minimum of -100,000, an invalid value that should be cleaned.
- No feature importance analysis yet; it would show which features (likely cibil_score) drive decisions.
- Evaluation uses a single train/test split. Cross-validated metrics, ROC-AUC and precision-recall curves would be more robust.
- Compare against Logistic Regression, XGBoost and Gradient Boosting; consider class_weight='balanced' or threshold tuning to reduce false approvals.
- Wrap preprocessing and model in a Pipeline and save it with joblib for deployment.

7. Tech Stack

Python, Pandas, NumPy, Matplotlib, Seaborn, scikit-learn (Google Colab)